

LLM Pipeline (LangChain + LangGraph) — Project Recipe Card

2 paradigms · 10-line agent · LCEL · StateGraph · tools · RAG · multi-agent · HITL · streaming · observability. Deep companion: llm_cheatsheet.md + llm_pipeline.ipynb

◆ The 2 paradigms (decide FIRST)

Work shape	Tool
Linear: input → transform → model → parse	LCEL (pipe)
RAG (retrieve → augment → generate)	LCEL
Few-shot · summarise · structured extract	LCEL
Tool-calling agent (loop until done)	LangGraph
Multi-agent · routing	LangGraph
HITL approval gates	LangGraph
Branching/long-running conversation	LangGraph

Rule: linear = LCEL. Conditional / iterative = LangGraph. Don't shoehorn a state machine into LCEL; don't wrap a 3-line RAG chain in a graph.

◆ LCEL pattern (4 steps, always the same)

```
chain = prompt | model | parser                # 1. COMPOSE: pipe
Runnables
chain.invoke({"input": ...})                    # 2. INVOKE: sync,
single shot
async for chunk in chain.astream({...}): ...    # 3. STREAM: async
tokens
chain.batch([ {...}, {...}, {...} ])           # 4. BATCH:
concurrent runs
# Every Runnable supports invoke / ainvoke / stream / astream /
batch / abatch
```

◆ LangGraph pattern (5 steps, always the same)

```
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.checkpoint.memory import InMemorySaver
```

```
# 1. STATE – shape that flows between nodes (TypedDict +
reducers)
class State(TypedDict):
    messages: Annotated[list, add_messages]
```

```
# 2. NODES – pure functions state -> state-update (dict)
def my_node(state: State) -> dict:
    return {"messages": [{"role": "assistant", "content":
"..."}]}
```

```
# 3. GRAPH – add nodes + edges + START/END
builder = StateGraph(State)
builder.add_node("step1", my_node)
builder.add_edge(START, "step1")
builder.add_edge("step1", END)
```

```
# 4. COMPILE with checkpoint (for memory + HITL)
graph = builder.compile(checkpointer=InMemorySaver())
```

```
# 5. RUN with thread id for persistence
config = {"configurable": {"thread id": "user-42"}}
out = graph.invoke({"messages": [{"role": "user", "content":
"hi"}]}, config)
```

Everything is a Runnable. Both graph and chain implement the same interface (invoke/stream/batch).

◆ Starter · 10-line tool-calling agent (LangChain 1.x)

```
# Canonical create agent (LangChain ≥1.3 + LangGraph ≥1.2)
from langchain.agents import create_agent
from langchain.tools import tool
```

```
@tool
def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's sunny in {city}"
```

```
agent = create_agent(
    model="anthropic:claude-sonnet-4-6",    # provider:model
    string
    tools=[get_weather],
    system_prompt="You are a helpful assistant.",
```

```
)

out = agent.invoke({"messages": [{"role": "user", "content":
"weather in Paris?"}]}])
print(out["messages"][-1].content)
```

API change: langgraph.prebuilt.create_react_agent (legacy) → langchain.agents.create_agent (1.x). **prompt= kwarg → system_prompt=.**

● Setup + env

Install (uv)

```
uv add langchain langgraph langsmith \
langchain-openai langchain-anthropic \
langchain-mcp-adapters chromadb \
pydantic tenacity
```

Required env vars

```
export OPENAI_API_KEY=sk-...
export ANTHROPIC_API_KEY=sk-ant-...
export LANGSMITH_API_KEY=lsv2...
export LANGSMITH_TRACING=true          # auto-traces ALL LC
runs
export LANGSMITH_PROJECT=my-project
```

Trace by default. Setting LANGSMITH_TRACING=true is the cheapest observability win in the ecosystem.

Version floors (1.x)

- langchain >= 1.3 — create_agent, unified imports
- langgraph >= 1.2 — Command, modern checkpoints
- langsmith >= 0.2 — datasets + LLM-as-judge
- Pin in pyproject.toml — ecosystem moves weekly

● Models — init + invoke

Provider-agnostic init

```
from langchain.chat_models import init_chat_model
```

```
# String form (recommended) – "provider:model"
m = init_chat_model("anthropic:claude-sonnet-4-6",
temperature=0)
m = init_chat_model("openai:gpt-4o", temperature=0)
m = init_chat_model("openai:gpt-4o-mini",
temperature=0.7, max_tokens=1024)
```

```
# Explicit class (alternative, more knobs)
from langchain.anthropic import ChatAnthropic
m = ChatAnthropic(model="claude-sonnet-4-6", temperature=0)
```

Invoke / stream

```
m.invoke("hi")                                # → AIMessage
m.invoke([{"role": "user", "content": "hi"}])
async for chunk in m.astream("hi"):
    print(chunk.content, end="", flush=True)
m.batch(["q1", "q2", "q3"])                    # concurrent
```

Structured output (typed)

```
from pydantic import BaseModel, Field
```

```
class Joke(BaseModel):
    setup: str
    punchline: str
    rating: int = Field(..., ge=1, le=10)
```

```
llm =
init_chat_model("openai:gpt-4o").with_structured_output(Joke)
joke = llm.invoke("Tell me a programming joke")
print(joke.setup, joke.punchline)              # already a Joke
instance
```

```
# Include raw AIMessage alongside parsed:
llm = m.with_structured_output(Joke, include_raw=True)
res = llm.invoke("..."); res["parsed"]; res["raw"]
Accepts Pydantic / TypedDict / JSON Schema. Pydantic = best UX.
```

● Messages

```
from langchain.messages import (
    SystemMessage, HumanMessage, AIMessage, ToolMessage)
```

```
msgs = [
    SystemMessage("You are concise."),
    HumanMessage("Summarise quantum tunnelling"),
]
m.invoke(msgs)
```

```
# OpenAI-style dicts also accepted everywhere:
m.invoke([{"role": "system", "content": "..."},
{"role": "user", "content": "..."},
{"role": "assistant", "content": "..."}])
```

Type	Use
SystemMessage	persona, rules, format requirements
HumanMessage	user input
AIMessage	model response (returned)
ToolMessage	tool result (paired with tool_call_id)

● Prompts (templates)

Chat template

```
from langchain_core.prompts import ChatPromptTemplate
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You translate {src} to {dst}."),
    ("human", "{text}"),
])
prompt.invoke({"src": "EN", "dst": "FR", "text": "hello"})
```

Few-shot

```
from langchain_core.prompts import (
    ChatPromptTemplate, FewShotChatMessagePromptTemplate)
examples = [
    {"q": "2+2", "a": "4"},
    {"q": "3*5", "a": "15"},
]
```

```
ex_prompt = ChatPromptTemplate.from_messages(
    [("human", "{q}"), ("ai", "{a}")]])
few = FewShotChatMessagePromptTemplate(
    examples=examples, example_prompt=ex_prompt)
final = ChatPromptTemplate.from_messages([
    ("system", "You answer arithmetic."), few, ("human",
"{q}")])
```

MessagesPlaceholder (history)

```
from langchain_core.prompts import MessagesPlaceholder
prompt = ChatPromptTemplate.from_messages([
    ("system", "..."),
    MessagesPlaceholder("history"),
    ("human", "{input}"),
])
```

```
prompt.invoke({"history": prior_msgs, "input": "..."})
```

▲ LCEL composition

The 5 building blocks	
Runnable	Use
RunnablePassthrough	identity; pass-through
RunnableLambda(fn)	wrap any callable
RunnableParallel({a:r1, b:r2})	fan-out, concurrent
RunnableBranch	if/else routing
RunnableSequence	what builds

Common pipe patterns

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough,
RunnableParallel
```

```
# Simple
chain = prompt | model | StrOutputParser()
```

```
# Parallel branches (single input, multiple outputs)
chain = RunnableParallel({
    "answer": prompt | model | StrOutputParser(),
    "language": detect_lang_chain,
```

```

})

# Pass extra context alongside the input
chain = (
    RunnablePassthrough.assign(context=retriever)
    | prompt | model | StrOutputParser()
)

# Make any function a step
chain = RunnableLambda(lambda x: x.upper()) | model

Modifiers
chain.with_config(tags=["prod"], metadata={"env": "v1"})
chain.with_retry(stop_after_attempt=3,
                wait_exponential_jitter=True)
chain.with_fallbacks([cheaper_model_chain])
chain.with_listeners(on_start=..., on_end=..., on_error=...)

```

▲ Tools (function calling)

@tool decorator (canonical)

```

from langchain.tools import tool
from pydantic import BaseModel, Field
import yfinance as yf

@tool
def get_price(symbol: str) -> float:
    """Return latest close price for the given ticker (USD)."""
    return float(yf.Ticker(symbol).history(period="1d")
["Close"].iloc[-1])

# Structured args via Pydantic for richer schemas:
class SearchInput(BaseModel):
    query: str = Field(..., description="search query")
    k: int = Field(default=5, ge=1, le=20)

@tool(args_schema=SearchInput)
def search(query: str, k: int = 5) -> list[str]:
    """Search the knowledge base."""
    return retriever.invoke(query)[:k]

```

Docstring becomes the tool description sent to the model — write it for the model.

Bind to model directly (no agent)

```

m_with_tools = m.bind_tools([get_price, search])
out = m_with_tools.invoke("price of AAPL?")
# out.tool_calls == [{"name": "get_price", "args":
{"symbol": "AAPL"}, "id": "..."}]
for tc in out.tool_calls:
    result = {"get_price": get_price, "search": search}
    [tc["name"]].invoke(tc["args"])
    # Feed result back as ToolMessage with matching id...

```

Error handling

```

from langchain_core.tools import ToolException

@tool
def risky():
    """Do something that might fail."""
    raise ToolException("retryable error msg")
# Agent will receive the exception message as a ToolMessage and
can recover.

```

▲ Agents — create_agent (1.x)

Canonical agent

```

from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model="anthropic:claude-sonnet-4-6", # or an instantiated
    model
    tools=[get_price, search],
    system_prompt="You answer using the supplied tools.",
    checkpointer=InMemorySaver(), # thread-scoped
    memory
)
config = {"configurable": {"thread_id": "user-42"}}
agent.invoke({"messages": [{"role": "user", "content": "q?"}]},
config)

```

Old API: create_react_agent(model, tools, prompt=...) → **New API:** create_agent(model, tools, system_prompt=...). The deprecated wrapper still works as of 1.3 but emits warnings.

Customising (1.x kwargs)

- response_format=PydanticModel — typed final answer (read via out["structured_response"], NOT messages)
- state_schema=MyState — extend the default messages-only state
- middleware=[...] — pre/post/around model hooks (1.x replacement for legacy hooks). See Middleware section.
- checkpointer=InMemorySaver() — thread-scoped memory + HITL prerequisite

API change in 1.x: interrupt_before=, pre_model_hook=, post_model_hook= are GONE. HITL + hooks are now configured via middleware=[...].

Stream agent steps

```

for ev in agent.stream({"messages": [...]}, config,
stream_mode="updates"):
    print(ev) # node-by-node state updates
# stream_mode: "values" (full state) / "messages" (tokens) /
"updates" (deltas)

```

▲ Middleware (the 1.x extensibility primitive)

```

from langchain.agents import create_agent
from langchain.agents.middleware import (
    HumanInTheLoopMiddleware, # HITL approval gates per
    tool
    SummarizationMiddleware, # context-window management
    ModelFallbackMiddleware, # auto-fallback to cheaper
    model
    # plus AnthropicPromptCachingMiddleware,
    PIIRedactionMiddleware, etc.
)

agent = create_agent(
    model="anthropic:claude-sonnet-4-6",
    tools=[get_price, search],
    system_prompt="You are a trading assistant.",
    middleware=[
        HumanInTheLoopMiddleware(interrupt_on={"get_price":
True}), # approve only get_price
        SummarizationMiddleware(model="openai:gpt-4o-mini",
max_tokens=4000),
        ModelFallbackMiddleware(fallback_model="openai:gpt-4o-
mini"),
    ],
    checkpointer=InMemorySaver(),
)

```

Use middleware for: HITL gates, context summarisation, model fallback, retries, prompt caching, PII redaction, dynamic prompts, dynamic tools. Compose multiple — order matters (outermost first).

▲ ToolNode (canonical tool executor for graphs)

```

from langgraph.prebuilt import ToolNode, tools_condition

tools = [get_price, search]
tool_node = ToolNode(tools) # parallel calls + error
handling + ToolMessage shaping
m_with_tools = m.bind_tools(tools)

def call_model(state):
    return {"messages":
[m_with_tools.invoke(state["messages"])]}

builder = StateGraph(MessagesState)
builder.add_node("agent", call_model)
builder.add_node("tools", tool_node)
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", tools_condition) #
built-in router
builder.add_edge("tools", "agent")
graph = builder.compile()

```

Use ToolNode in custom graphs; create_agent wires it for you.

● LangGraph StateGraph (custom)

Built-in reducer

```

from langgraph.graph import MessagesState
# MessagesState = TypedDict with messages: Annotated[list,
add_messages]
# Use directly if your state is just messages.

```

Custom state with reducers

```

from typing import Annotated, TypedDict
import operator

class State(TypedDict):
    messages: Annotated[list, add_messages] # append + dedupe
    by ID
    scratch: Annotated[list[str], operator.add] # plain
    concat
    count: int # plain
    overwrite

Annotated[T, reducer] tells LangGraph how to merge concurrent updates.

```

Conditional edges

```

def router(state: State) -> str:
    return "yes" if needs_tool(state) else "no"

builder.add_conditional_edges(
    "agent", router, {"yes": "tools", "no": END})

# Or: return list of next nodes (parallel fan-out)
def fan_out(state) -> list[str]: return ["a", "b"]
builder.add_conditional_edges("split", fan_out, ["a", "b"])

```

Visualise

```

from IPython.display import Image
Image(graph.get_graph().draw_mermaid_png())
# Or text: print(graph.get_graph().draw_mermaid())

```

▲ Multi-agent (Command + supervisor)

Command (state-update + routing)

```

from langgraph.types import Command
from typing import Literal

def supervisor(state) ->
Command[Literal["researcher", "writer", "__end__"]]:
    decision = decide_next(state)
    return Command(
        update={"messages": [...]}, # state update
        (optional)
        goto=decision, # next node –
        overrides static edges
    )
# Use in node return; with conditional edges; or to resume an
interrupt
# (resume= is INPUT-only to invoke/stream – don't conflate with
goto/update).

```

Supervisor pattern

```

researcher = create_agent(model, tools=[search],
system_prompt="research")
writer = create_agent(model, tools=[],
system_prompt="write")

class State(MessagesState):
    next: str

def supervisor(state):
    # LLM decides next worker
    return Command(goto="researcher" if needs_data(state) else
"writer")

builder = StateGraph(State)
builder.add_node("supervisor", supervisor)
builder.add_node("researcher", researcher)
builder.add_node("writer", writer)
builder.add_edge(START, "supervisor")
builder.add_edge("researcher", "supervisor") # loop back to
supervisor
builder.add_edge("writer", END)
graph = builder.compile(checkpointer=InMemorySaver())

```

```
◆ HITL — interrupt + resume
from langgraph.types import interrupt, Command

def review(state):
    decision = interrupt({"propose": state["draft"]}) # pauses
    execution
    return {"messages": [{"role": "user", "content": decision}]}

config = {"configurable": {"thread_id": "1"}}
# Run hits interrupt; returned state contains __interrupt__:
out = graph.invoke({"draft": "..."}, config)

# Resume with user's answer:
graph.invoke(Command(resume="approved"), config)
Requires checkpoint. Interrupt persists state to the checkpoint; resume
hydrates it. No checkpoint = no HITL.
Don't confuse: Command(resume=...) = INPUT to invoke/stream.
Command(update=..., goto=...) = RETURN from a node. They are different
roles of the same class.
```

```
◆ Memory + persistence
Short-term (within a thread)
from langgraph.checkpoint.memory import InMemorySaver #
dev
from langgraph.checkpoint.sqlite import SqliteSaver #
local prod
from langgraph.checkpoint.postgres import PostgresSaver #
cloud prod

graph = builder.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "user-42"}}
graph.invoke({"messages": [...]}, config) # state saved at
thread "user-42"
graph.invoke({"messages": [...]}, config) # auto-loads prior
state
```

MemorySaver was renamed InMemorySaver. Old name still works (deprecated).

```
Long-term (across threads — Store API)
from langgraph.store.memory import InMemoryStore
from langgraph.store.base import BaseStore

store = InMemoryStore()
def remember_node(state, *, store: BaseStore):
    user_id = state["user_id"]
    store.put(("memories", user_id), key="profile",
value={"role": "trader"})
    # Retrieve:
    mem = store.get(("memories", user_id), key="profile")
    return {"messages": [...]}

graph = builder.compile(checkpointer=InMemorySaver(),
store=store)
```

```
◆ RAG patterns
Basic RAG (LCEL)
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import
RecursiveCharacterTextSplitter
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

def format_docs(docs): # define before use!
    return "\n\n".join(d.page_content for d in docs)

splits = RecursiveCharacterTextSplitter(chunk_size=800,
chunk_overlap=100
).split_documents(docs)
vs = Chroma.from_documents(splits, OpenAIEmbeddings())
retriever = vs.as_retriever(search_kwargs={"k": 4})

rag = (
    {"context": retriever | format_docs, "question":
RunnablePassthrough()}
    | prompt | model | StrOutputParser())
```

```
)
rag.invoke("How does X work?")
RAG with citations (structured output)
class Answer(BaseModel):
    answer: str
    sources: list[str] # doc ids cited

rag_struct = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt | model.with_structured_output(Answer)
)
```

Advanced retrieval

Pattern	Win
Multi-query (LLM rewrites)	+recall for ambiguous q
Parent-document	embed small / return large
Reranker (Cohere/Voyage)	+precision on top-k
HyDE (hypothetical doc)	q rewrite for sparse corpora
Self-query	LLM emits structured filter

Legacy retrievers moved to langchain_classic in 1.x:
from langchain_classic.retrievers import MultiQueryRetriever,
ParentDocumentRetriever
from langchain_classic.retrievers.contextual_compression import
ContextualCompressionRetriever
from langchain_cohere import CohereRerank # uv add
langchain-cohere
rr = ContextualCompressionRetriever(
 base_retriever=retriever, base_compressor=CohereRerank())

Retriever as a tool (for agents)
from langchain.tools.retriever import create_retriever_tool
search_tool = create_retriever_tool(retriever, name="kb_search",
description="Search
internal KB")
agent = create_agent(model, [search_tool], system_prompt="Use
kb_search.")

```
◆ Streaming
stream_mode Yields
"values" full state after each node
"updates" just the delta per node
"messages" token-by-token (LLM chunks)
"custom" via config['callbacks']
"debug" verbose internal events

async for chunk in graph.astream(inp, config,
stream_mode="messages"):
    if isinstance(chunk[0], AIMessageChunk):
        print(chunk[0].content, end="", flush=True)

# Stream multiple modes at once:
for ev in graph.stream(inp, config,
stream_mode=["updates", "messages"]):
    mode, payload = ev # tuple form when list of modes
```

```
◆ Middleware + reliability
RunnableConfig (settings propagation)
cfg = {"configurable": {"thread_id": "1"},
      "tags": ["prod"],
      "metadata": {"user": "x"},
      "max_concurrency": 5,
      "recursion_limit": 25}
chain.invoke(inp, config=cfg)
Caching (skip identical LLM calls)
from langchain.globals import set_llm_cache
from langchain_community.cache import InMemoryCache, SQLiteCache
set_llm_cache(SQLiteCache(database_path=".llm_cache.db"))
Rate limit (provider quota)
from langchain_core.rate_limiters import InMemoryRateLimiter
limiter = InMemoryRateLimiter(requests_per_second=2,
                              check_every_n_seconds=0.1,
                              max_bucket_size=10)
m = init_chat_model("openai:gpt-4o", rate_limiter=limiter)
Retry + fallback
```

```
from openai import RateLimitError # provider-specific; or
anthropic.RateLimitError
resilient = (
    chain.with_retry(stop_after_attempt=3,
                    wait_exponential_jitter=True,
                    retry_if_exception_type=(RateLimitError,))
    .with_fallbacks([cheaper_chain])
)
Callbacks
from langchain_core.callbacks import BaseCallbackHandler

class CostTracker(BaseCallbackHandler):
    def on_llm_end(self, response, **k):
        # Provider-agnostic: use AIMessage.usage_metadata (set
        on every chat model).
        for gen in response.generations:
            usage = getattr(gen[0].message, "usage_metadata",
None)
            if usage:
                log.info(f"in={usage['input_tokens']}")
out={usage['output_tokens']} } "
                f"total={usage['total_tokens']}")

chain.invoke(inp, config={"callbacks": [CostTracker()]})
```

```
◆ LangSmith (observability + eval)
Auto-trace (env-var only)
# Already set above:
# LANGSMITH_API_KEY, LANGSMITH_TRACING=true, LANGSMITH_PROJECT
# Every invoke/stream now writes a trace to the LangSmith UI.

# Add per-run tags / metadata:
chain.invoke(inp, config={"tags": ["prod", "v2"],
                          "metadata": {"user_id": "42"}})
```

Evaluation

```
from langsmith import Client
from langsmith.evaluation import evaluate

client = Client()
# 1. Create dataset (one-time)
ds = client.create_dataset("qa_v1")
client.create_examples(
    inputs=[{"q": "what is X?"}, ...],
    outputs=[{"a": "X is ..."}, ...],
    dataset_id=ds.id)

# 2. Run evaluation with built-in + LLM-as-judge evaluators
def my_evaluator(run, example):
    pred = run.outputs["answer"]; gold = example.outputs["a"]
    return {"key": "exact_match", "score": int(pred.strip() ==
gold.strip())}

results = evaluate(
    lambda inp: rag.invoke(inp["q"]),
    data="qa_v1",
    evaluators=[my_evaluator],
    experiment_prefix="rag-v1",
)
```

```
◆ Deployment (FastAPI)
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from langgraph.checkpoint.postgres import AsyncPostgresSaver
import json

app = FastAPI()

class Req(BaseModel):
    q: str
    thread_id: str

# Async-first: agent.ainvoke / agent.astream do NOT block the
event loop.
@app.post("/invoke")
async def invoke(r: Req):
```

```
    cfg = {"configurable": {"thread_id": r.thread_id}}
    out = await agent.ainvoke({"messages":
[{"role":"user","content": r.q}], cfg)
    return {"answer": out["messages"][-1].content}

@app.post("/stream")
async def stream(r: Req):
    cfg = {"configurable": {"thread_id": r.thread_id}}
    async def gen():
        async for ev in agent.astream(
            {"messages":[{"role":"user","content": r.q}],
            cfg, stream_mode="messages"):
            chunk = ev[0]
            yield f"data: {json.dumps({'chunk': chunk.content})}"
        \n\n"
    return StreamingResponse(gen(), media_type="text/event-
stream")
```

Production checklist: async endpoints + ainvoke/astream; load agent at FastAPI lifespan (not import-time); use AsyncPostgresSaver for cross-process checkpointing; set recursion_limit + per-request timeouts; auth BEFORE trusting thread_id.

- ▲ **Top traps (6-month memory)**
- **create_react_agent is legacy.** Use langchain.agents.create_agent; prompt= → system_prompt=.
- **MemorySaver renamed** → InMemorySaver. Old name deprecated.
- **set_entry_point/set_finish_point legacy.** Use add_edge(START, ...) / add_edge(..., END).
- **Command roles:** resume= is INPUT to invoke; update= + goto= are RETURN from a node. Don't conflate.

- **No checkpointer = no HITL, no memory.** Add one at compile time.
- **Tools without docstrings** → model can't see what they do; treat docstring as the model's prompt.
- **Streaming mode mismatch:** "messages" = tokens; "updates" = node deltas; "values" = full state. Pick deliberately.
- **Forgetting thread_id** → fresh state every call; memory invisible.
- **State schema:** prefer TypedDict; use Pydantic only when you need validation at node boundaries.
- **response_format= result** lives at out["structured_response"], NOT in out["messages"][-1]. Easy silent miss.
- **llm_output is provider-specific (often None);** prefer msg.usage_metadata for token counts.
- **Sync FastAPI def with agent.invoke** blocks the event loop. Use async def + await agent.ainvoke(...).
- **Caching with structured output:** cache key is the prompt — same prompt + different schema returns stale.
- **No tracing** → debugging is guessing. Set LANGSMITH_TRACING=true as table-stakes.
- **Recursion limit** defaults to 25 — agent loops can silently truncate. Raise via config["recursion_limit"].

◆ Quick-reference (recall in 5 seconds)	
Task	Canonical pattern
Init model	init_chat_model("provider:model", temperature=0)
Structured output	model.with_structured_output(PydanticClass)
Tool	@tool on function with docstring + typed args
LCEL pipe	prompt model StrOutputParser()
RAG	

	{"context": retriever, "question": passthrough} prompt model
10-line agent	create_agent(model="...", tools=[...], system_prompt="...")
State graph	StateGraph(State) → add_node → add_edge(START, ...) → compile(checkpointer=)
State reducer	Annotated[list, add_messages] (built-in)
Routing + update	return Command(update=..., goto=...) from a node
HITL	interrupt(payload) inside node; Command(resume=v) to i nvoke
Short-term memory	checkpointer + config["configurable"]["thread_id"]
Long-term memory	InMemoryStore / PostgresStore + store.put / get
Stream tokens	graph.astream(inp, cfg, stream_mode="messages")
Trace	LANGSMITH_TRACING=true in env
Eval	langsmith.evaluation.evaluate(fn, data=ds, evaluators=[...])
Retry	chain.with_retry(stop_after_attempt=3, wait_exponential_jitter=True)
Fallback	chain.with_fallbacks([cheaper_chain])
Rate limit	InMemoryRateLimiter(requests_per_second=N) passed to model
Cache	set_llm_cache(SQLiteCache(...))
Visualise	graph.get_graph().draw_mermaid_png()